MovieLens Explorer: An Explainable Hybrid Recommendation System Using Relational and Graph Data

Chihong Yip

March 11, 2026

Abstract

MovieLens Explorer is an explainable movie recommendation and discovery tool built on the MovieLens Latest Small dataset. The project combines a relational database layer for structured storage and analytics with a graph-based relationship layer for similarity search, local movie exploration, and recommendation explanation. The final system demonstrates how multiple data structures can be integrated into a single interactive application. Rather than returning only a ranked list of recommended movies, the tool also provides interpretable evidence such as shared genres, similar users, and related tags.

GitHub Repository: `https://github.com/YipGitGIT/movielens-explorer`

1. Introduction

Recommendation systems are useful only if users can understand and trust their outputs. In many applications, the system returns suggestions but does not explain why those suggestions appear. This makes the recommendation process difficult to interpret and reduces user trust.

This project addresses that problem by building **MovieLens Explorer**, an explainable movie recommendation and discovery tool based on the MovieLens Latest Small dataset. The main goal of the tool is to combine a **relational database element** with a **graph data element** in a single application. The relational layer supports structured storage, joins, filtering, and aggregate analytics. The graph layer supports neighborhood exploration, user similarity, movie similarity, and interpretable recommendation evidence.

The final application is implemented in Python with Streamlit as the interface, SQLite as the relational backend, and a graph-based relationship layer built with NetworkX-style modeling.

The result is a complete interactive application that demonstrates what the tool does, why the database choices were made, how those structures fit the purpose of the tool, and how the work was completed.

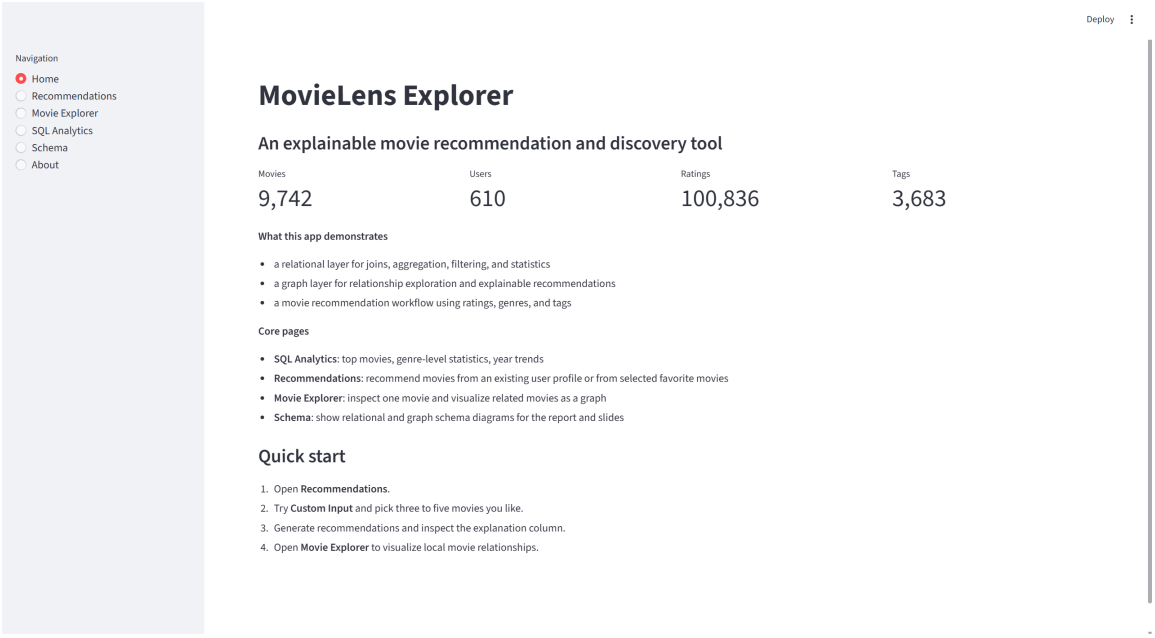


Figure 1: Home page of MovieLens Explorer. The interface summarizes the dataset and the main functions of the tool.

2. Project Goal and Main Use Cases

The project goal is to create an application that supports movie recommendation, movie discovery, and database-driven analysis while clearly showing the role of both the relational and graph structures.

The main use cases are:

- personalized movie recommendation,
- explanation of recommendation evidence,
- similar-user discovery,
- related-movie exploration,
- genre-level and release-year analytics.

These use cases were chosen because they naturally require both structured tabular queries and relationship-centered reasoning. Recommendation and analytics depend on relational

joins and aggregation, while explanation and movie-neighborhood exploration depend on graph-style relationships.

3. Dataset Description

The project uses the **MovieLens Latest Small** dataset. This dataset contains movie rating and tagging activity collected from the MovieLens platform. It includes:

- 100,836 ratings,
- 3,683 tag applications,
- 9,742 movies,
- 610 users.

The data is stored in four CSV files:

- `movies.csv`: movie ID, title, and genres,
- `ratings.csv`: user ID, movie ID, rating, and timestamp,
- `tags.csv`: user ID, movie ID, free-text tag, and timestamp,
- `links.csv`: movie ID, IMDb ID, and TMDB ID.

This dataset is appropriate for the project for three reasons. First, it contains clear structured entities such as users, movies, ratings, tags, and genres, which supports normalized relational design. Second, it contains natural relationships between entities, which supports graph-based modeling. Third, the dataset is large enough to support meaningful recommendation and analytics examples, but small enough to manage within a course project setting.

The dataset also has limitations. It does not include demographic user information, detailed movie plots, directors, cast information, or explicit social-network relationships between users. As a result, the graph relationships in this project are behavior-derived rather than true social links.

4. Technology Choices and Design Rationale

The system was designed to balance clarity, implementation speed, and alignment with the project requirements.

I chose **Streamlit** as the application layer because it allows rapid development of an interactive interface with multiple pages, tables, sliders, and plots. This made it possible to present recommendation results, analytics, and schema visualization in a single tool.

I chose **SQLite** for the relational layer because the MovieLens Latest Small dataset is structured, moderate in size, and well suited for a lightweight SQL database. SQLite supports normalized schema design, joins, filtering, and aggregate queries without additional server setup. Since the project goal is to demonstrate relational design rather than large-scale deployment, SQLite is an appropriate choice.

I chose a **graph-based modeling layer** implemented in Python because the recommendation problem contains relationship-centered structure. Users connect to movies through ratings, movies connect to genres and tags, and movies can also connect to other movies through derived similarity. A graph representation makes these relationships easier to reason about and visualize than a purely relational approach.

This hybrid design was chosen intentionally. The relational layer handles clean structured storage and aggregate analysis, while the graph layer handles similarity, neighborhood exploration, and recommendation explanation. The combination matches the strengths of each structure to the tasks required by the tool.

5. Application Overview

MovieLens Explorer is designed as an interactive recommendation and discovery system. The application contains six main pages:

- **Home**: tool summary and dataset overview,
- **Recommendations**: recommendation generation from either an existing user or custom movie input,
- **Movie Explorer**: local movie-neighborhood exploration,
- **SQL Analytics**: descriptive statistics and temporal trends,
- **Schema**: relational and graph schema visualization,
- **About**: project framing and problem statement.

The recommendation page supports two interaction modes:

1. **Existing User Mode**: select a user already present in the dataset.

2. **Custom Input Mode**: select several favorite movies and construct a preference profile.

This design improves usability. Existing-user mode demonstrates personalized recommendation from real historical data, while custom-input mode makes the tool accessible even without a known MovieLens user ID.

6. Relational Database Design

The relational component of the application is implemented in **SQLite**. This layer is responsible for normalized storage and efficient structured queries. The schema contains the following tables:

- `users(user_id)`
- `movies(movie_id, title, year)`
- `genres(genre_id, genre_name)`
- `movie_genres(movie_id, genre_id)`
- `ratings(user_id, movie_id, rating, timestamp)`
- `tags(user_id, movie_id, tag, timestamp)`
- `links(movie_id, imdb_id, tmdb_id)`

This design separates movies, genres, and user-generated interactions into different tables. The bridge table `movie_genres` resolves the many-to-many relationship between movies and genres. The table `ratings` acts as the main fact table for recommendation and analytics, while `tags` stores additional user-generated metadata that improves interpretability.

The relational layer is used for:

- retrieving user rating histories,
- aggregating ratings by movie,
- computing average ratings and rating counts,
- summarizing genre-level statistics,
- extracting release-year trends,
- supporting table-based views in the application.

This part of the system is especially effective for tasks that require joins, filtering, ranking, or aggregation.

7. Data Loading and Database Construction

The raw MovieLens data is provided in CSV format. The first implementation step was to load `movies.csv`, `ratings.csv`, `tags.csv`, and `links.csv` into Python using pandas. After loading the raw files, the data was cleaned and reformatted before insertion into the relational database.

For the `movies` table, the release year was extracted from the title string and stored as a separate attribute. This makes release-year analysis possible without repeatedly parsing the title text during queries.

For genres, the original MovieLens format stores multiple genres in one pipe-separated string. To normalize this structure, genres were split into individual values and loaded into two relational tables: `genres` and `movie_genres`. This converts the raw genre field into a many-to-many SQL design that is more appropriate for joins and aggregation.

The `ratings` and `tags` files were then renamed into database-friendly attribute names and inserted into SQLite. Primary keys and indexes were added to support efficient filtering by user, movie, and genre. In particular, indexes on `ratings(user_id)`, `ratings(movie_id)`, and `movie_genres` improve the speed of recommendation and analytics queries.

After the relational layer was constructed, additional support objects were built in Python for graph-style analysis. These included:

- movie ID to title mappings,
- movie ID to genre mappings,
- user ID to rating-history mappings,
- movie ID to rating-history mappings,
- movie ID to tag collections.

The work was therefore completed in two stages:

1. normalized storage and querying in SQLite,
2. graph-style relationship construction on top of the cleaned data.

8. Graph Data Design

The graph component of the application models relationships between users, movies, genres, and tags. Although this project does not use an external graph database server, it uses a

clear graph schema and graph operations to satisfy the graph-data requirement.

The graph schema contains the following conceptual nodes:

- `User`
- `Movie`
- `Genre`
- `Tag`

The main edges are:

- `User` $\rightarrow Movie via$`RATED`,
- `Movie` $\rightarrow Genre via$`IN_GENRE`,
- `Movie` $\rightarrow Tag via$`HAS_TAG`,
- `Movie` $\rightarrow Movie via$`SIMILAR_TO`.

The graph layer is used for:

- movie neighborhood exploration,
- similar-user discovery,
- recommendation explanation,
- local relationship visualization.

Derived Graph Construction

Because the MovieLens dataset does not provide an explicit social graph, the graph layer was constructed from behavioral relationships. A `User--Movie` edge exists whenever a user rated a movie. A `Movie--Genre` edge exists whenever a movie belongs to a genre. A `Movie--Tag` edge exists whenever at least one user applied a tag to that movie.

The most important derived relationship is the `Movie--Movie` similarity edge. This edge is not directly stored in the dataset and must be computed. For a selected movie, the system compares it with other movies using three signals:

- overlap in users who gave high ratings,
- number of shared genres,
- number of shared tags.

Similarly, `User--User` similarity is derived from overlapping movie ratings. If two users have rated enough of the same movies, the system compares their centered rating vectors and computes a cosine-style similarity score. This produces a neighborhood of similar users, which is then used in recommendation.

This derived-graph construction is central to the project because it gives the graph element a real purpose. The graph structure is not included only to satisfy the requirement; it is what enables explainable recommendation, local neighborhood visualization, and related-movie exploration.

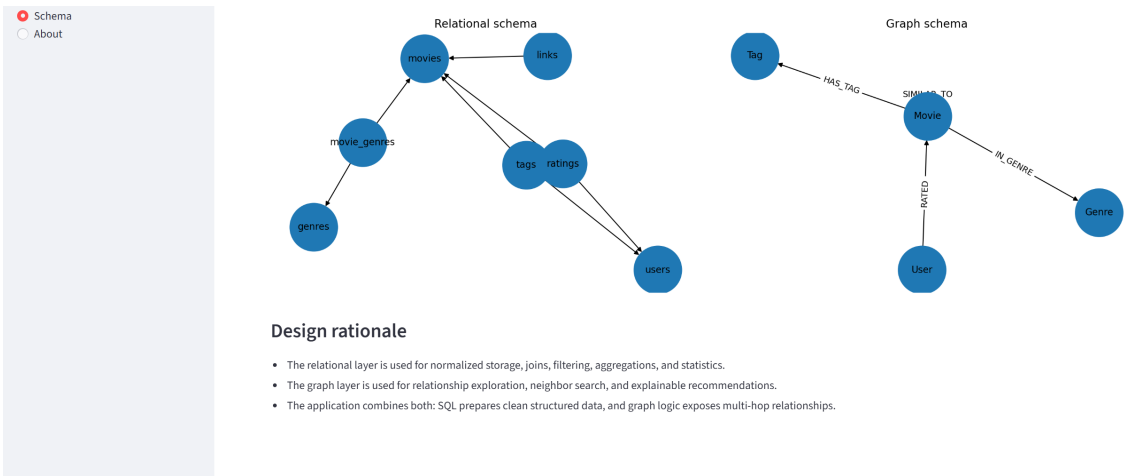


Figure 2: Relational schema and graph schema used in the application. The left diagram summarizes the relational tables and dependencies; the right diagram summarizes the graph entities and edges.

9. System Workflow

The application combines the relational and graph layers in a single recommendation pipeline.

Step 1: Input Construction

The user either selects an existing user profile or chooses several favorite movies in custom-input mode. In the second case, the application builds a preference profile by assigning a high assumed rating to the selected movies.

Step 2: Structured Data Retrieval

The relational layer retrieves movie metadata, user histories, genre summaries, and aggregate rating statistics. This step provides clean structured information that supports the rest of the pipeline.

Step 3: Similarity and Candidate Generation

The graph layer identifies similar users or related movies. Candidate recommendation items are generated from movies that similar users liked but the target profile has not rated yet.

Step 4: Scoring and Ranking

Each candidate movie receives a heuristic score based on:

- similarity-weighted ratings from neighboring users,
- overlap with the user's preferred genres,
- a small popularity adjustment based on the number of ratings.

Step 5: Explanation

The final output includes not only ranked movies but also an explanation column. This explanation highlights evidence such as:

- shared genres,
- similar users who liked the movie,
- top supporting users,
- representative tags.

10. Use Cases and How the Databases Are Utilized

This project is best understood through its main application use cases. Each use case relies on both the relational and graph layers, but the role of each layer differs depending on the task.

Use Case 1: Recommendation from an Existing User

In this use case, the user selects a MovieLens user ID already stored in the dataset. The relational layer retrieves that user's rating history and computes summary statistics such as favorite genres. The graph layer then identifies similar users based on overlapping ratings. Candidate movies are collected from those similar users, filtered to exclude already seen movies, and ranked into a recommendation list. Finally, the explanation column summarizes why each movie was recommended.

Use Case 2: Recommendation from Custom Favorite Movies

In this use case, the application supports a new-user scenario. The user selects several movies that they like, and the system creates a small preference profile by assigning high assumed ratings to those movies. The relational layer retrieves metadata and genre information for the selected movies. The graph layer compares that constructed profile with existing users in the dataset, identifies similar users, and returns movies that fit the inferred taste profile.

Use Case 3: Exploring a Movie Neighborhood

In this use case, the user selects one movie and inspects related movies. The relational layer retrieves the selected movie's metadata, rating count, average rating, genres, and tags. The graph layer then constructs a local neighborhood by linking the movie to other movies with shared genres, shared tags, or overlapping high-rating audiences. The result is both a ranked table of related movies and a visual graph centered on the selected title.

Use Case 4: Analytics and Trend Exploration

In this use case, the user inspects top movies, genre-level statistics, and release-year trends. This task is primarily driven by the relational layer because it depends on grouping, filtering, and aggregation. However, this analytics page complements the graph layer by giving a global summary of the same dataset used in recommendation and exploration.

11. Implementation Details and Scoring Logic

The recommendation method in this project was designed to be simple, transparent, and easy to explain. Rather than using a complex black-box model, the system uses a heuristic hybrid score that combines user similarity, genre overlap, and a small popularity adjustment.

For user similarity, the system compares two users only on movies both users rated. If the overlap is too small, similarity is set to zero. Otherwise, ratings are mean-centered on the shared movies and compared using a cosine-style similarity:

$$\mathrm{sim}(u,v) = \frac{\sum_{m \in M_{uv}} (r_{um} - \bar{r}_u)(r_{vm} - \bar{r}_v)}{\sqrt{\sum_{m \in M_{uv}} (r_{um} - \bar{r}_u)^2} \sqrt{\sum_{m \in M_{uv}} (r_{vm} - \bar{r}_v)^2}}$$

where M_{uv} is the set of movies rated by both users.

After similar users are identified, candidate movies are collected from movies that those users rated highly. For each candidate movie, the application computes a heuristic recommendation score of the form

$$\mathrm{score}(m) = \frac{\sum_{v \in N(u)} \mathrm{sim}(u,v) \cdot r_{vm} + \text{genre bonus}(m) + \text{popularity bonus}(m)}{\sum_{v \in N(u)} \mathrm{sim}(u,v)}$$

where $N(u)$ is the set of similar users considered for the target profile.

The movie-neighborhood score is also heuristic. A candidate related movie receives higher weight when it shares high-rating users, genres, or tags with the selected movie. These choices were made because the system needed to remain interpretable and directly connected to the available data.

This scoring method was chosen for two reasons. First, it is easy to inspect and explain. Second, it makes it straightforward to generate recommendation evidence such as shared genres and support from similar users. The goal of the project is not to optimize benchmark accuracy, but to build a recommendation system whose logic can be understood by a user and clearly described in a report.

12. Recommendation Component

The recommendation page demonstrates the main functionality of the project. In existing-user mode, the application loads the rating history of a selected user and computes that user's favorite genres. It then finds similar users and recommends unseen movies.

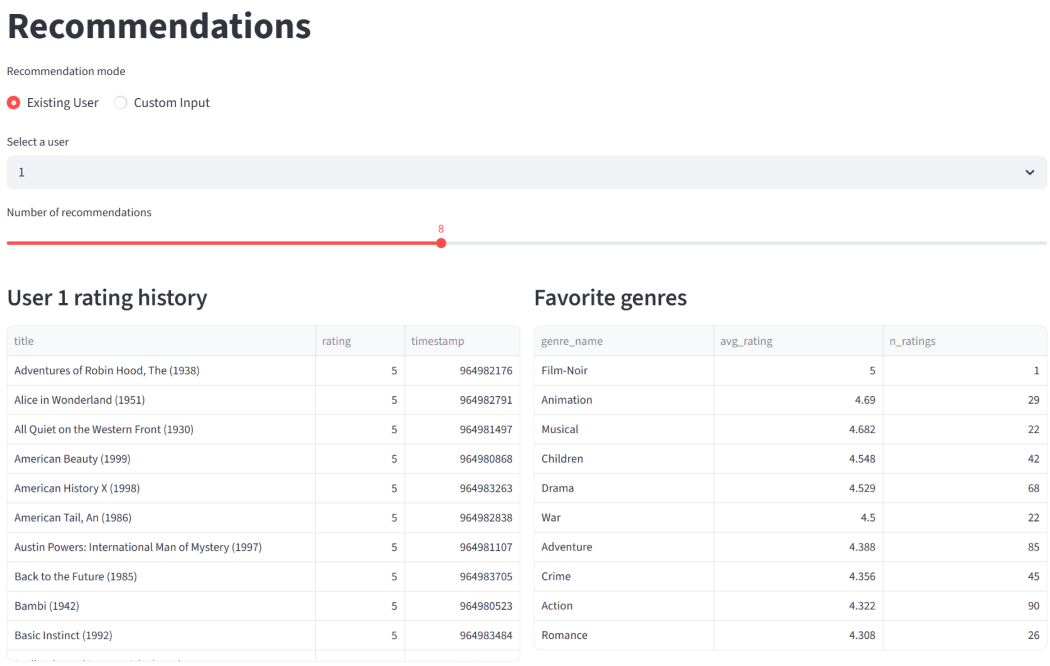


Figure 3: Recommendation page in existing-user mode. The application shows the selected user's rating history and favorite genres before generating recommendations.

The recommendation results include a ranked table with predicted score, average rating, rating count, and explanation. This design is important because the project goal is not only recommendation, but recommendation with interpretable evidence.

Deploy

Recommended movies

movieId	title	predicted_score	avg_rating	n_ratings	reason
91542	Sherlock Holmes: A Game of Shadows (2011)	45.776	3.763	38	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: Use
6016	City of God (Cidade de Deus) (2002)	43.717	4.147	75	shared genres: Action, Adventure, Crime | liked by 1 similar users | top supporters: User 3
117646	Dragonheart 2: A New Beginning (2000)	42.853	4.5	1	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: Use
6503	Charlie's Angels: Full Throttle (2003)	40.987	2.37	27	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: Use
380	True Lies (1994)	39.974	3.497	178	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: Use
1215	Army of Darkness (1993)	39.439	4.039	51	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: Use
145	Bad Boys (1995)	38.123	3.245	51	shared genres: Action, Comedy, Crime | liked by 1 similar users | top supporters: User 162
53125	Pirates of the Caribbean: At World's End (2007)	37.604	3.438	56	shared genres: Action, Adventure, Comedy | liked by 1 similar users | top supporters: Use

Most similar users

userId	similarity	overlap
550	1	3
1	1	232
473	0.9623	4
511	0.9258	6
9	0.9186	5
13	0.8783	6
366	0.8729	5
401	0.8669	5
535	0.8664	6
154	0.866	3

Figure 4: Recommendation results and similar-user table. Each recommendation includes evidence such as shared genres and similar-user support.

The custom-input mode extends the same logic to a new-user scenario. Instead of relying on a stored profile, the user chooses several favorite movies, and the system builds a preference profile from those selections. This is useful because it allows someone with no dataset user ID to interact with the tool directly.

13. SQL Analytics

The SQL Analytics page highlights the strengths of the relational layer. It provides descriptive summaries that are difficult to replace with a graph-only design.

The first table ranks movies by average rating subject to a minimum-rating threshold. This prevents little-known movies with only a few ratings from dominating the ranking. The second table summarizes average rating, total rating count, and number of movies for each genre.

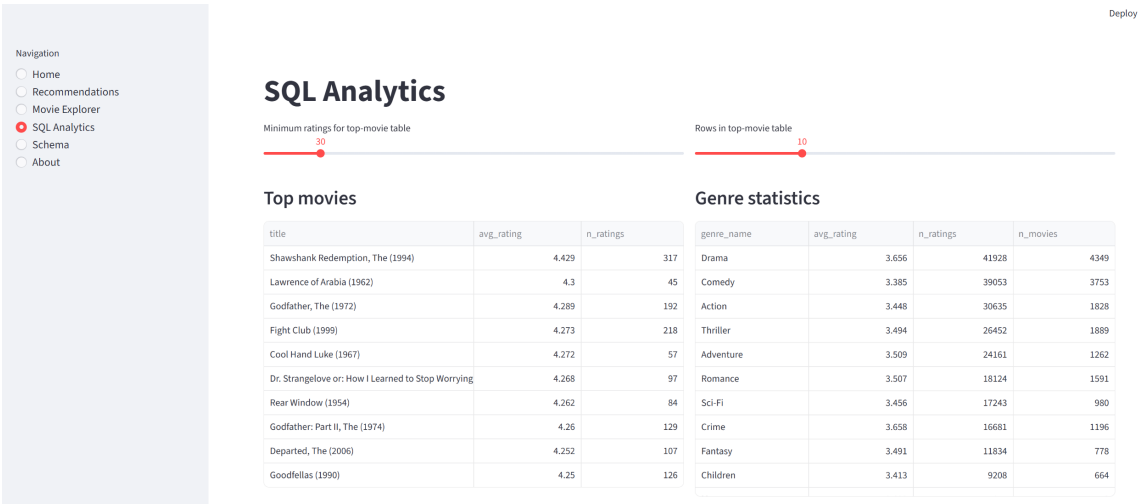


Figure 5: SQL Analytics page showing top movies and genre-level statistics generated from aggregate relational queries.

The bar chart below shows the most frequently rated genres. This gives a compact summary of where most user activity occurs in the dataset.

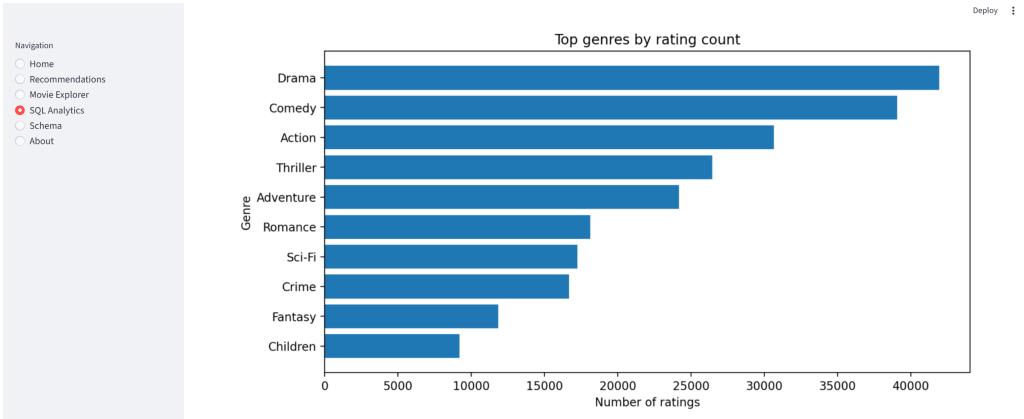


Figure 6: Top genres by rating count. Drama, Comedy, and Action dominate the dataset in total user rating activity.

The line chart below shows how average rating changes by release year. This provides a temporal summary of movie reception and demonstrates another task that is naturally handled by the relational layer.

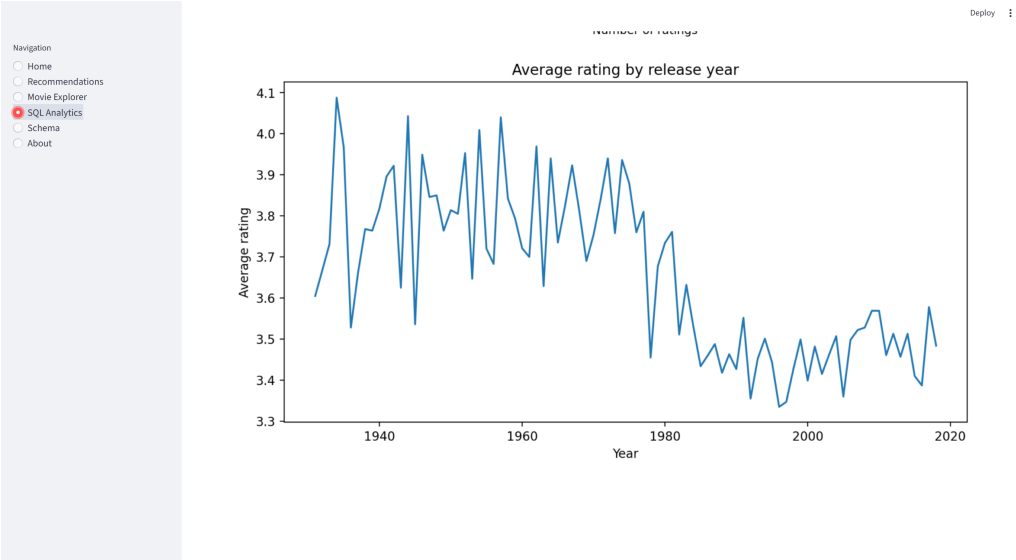
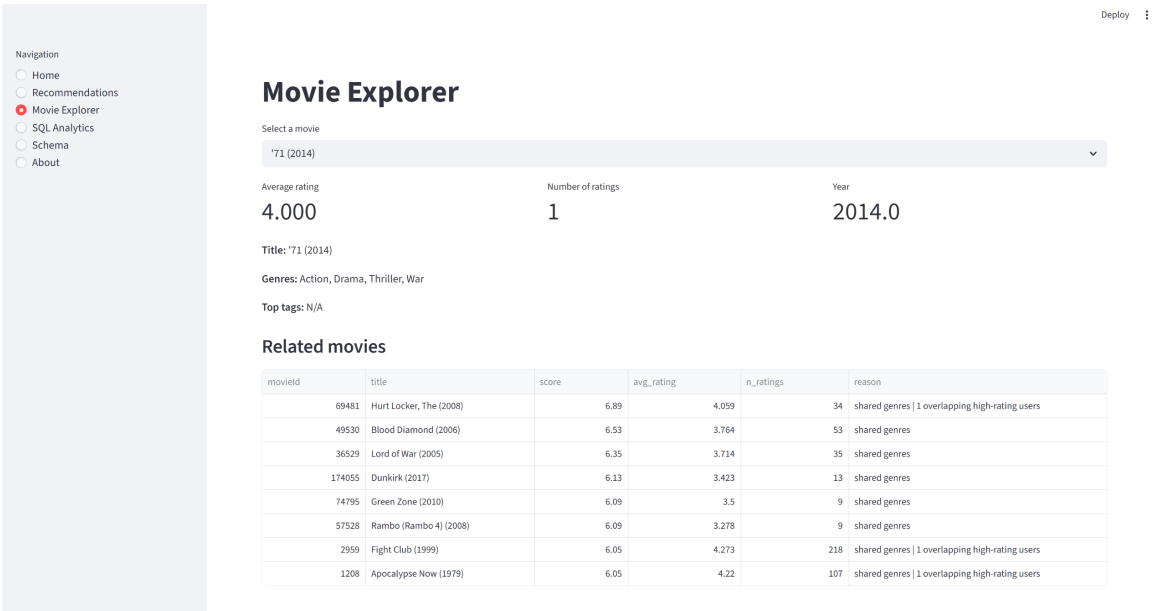


Figure 7: Average rating by release year. This trend view is generated from grouped SQL queries after extracting release year from movie titles.

14. Movie Explorer

The Movie Explorer page focuses on one selected movie and builds a local neighborhood around it. For each chosen movie, the application displays average rating, number of ratings, year, genres, and representative tags. It also returns a table of related movies together with a similarity score and a short explanation.



movieId	title	score	avg_rating	n_ratings	reason
69481	Hurt Locker, The (2008)	6.89	4.059	34	shared genres | 1 overlapping high-rating users
49530	Blood Diamond (2006)	6.53	3.764	53	shared genres
36529	Lord of War (2005)	6.35	3.714	35	shared genres
174055	Dunkirk (2017)	6.13	3.423	13	shared genres
74795	Green Zone (2010)	6.09	3.5	9	shared genres
57528	Rambo (Rambo 4) (2008)	6.09	3.278	9	shared genres
2959	Fight Club (1999)	6.05	4.273	218	shared genres | 1 overlapping high-rating users
1208	Apocalypse Now (1979)	6.05	4.22	107	shared genres | 1 overlapping high-rating users

Figure 8: Movie Explorer page. The application displays movie metadata and a related-movie table based on graph-style similarity signals.

The local graph visualization makes the relationship structure explicit. Instead of reading only a table, the user can see the selected movie at the center of its neighborhood and inspect nearby related movies. This is a clear example of why a graph-based data element is valuable in the application.

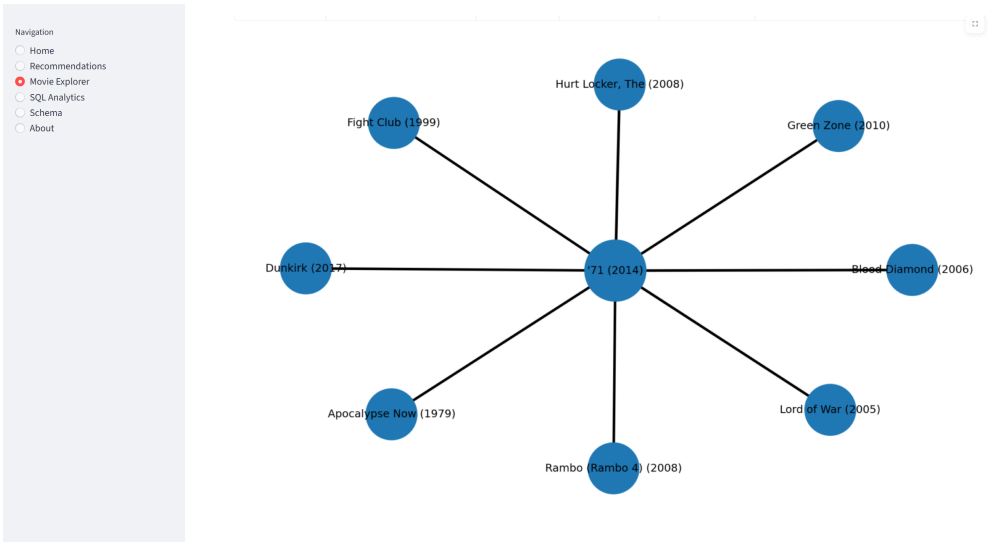


Figure 9: Local movie neighborhood graph. The selected movie is placed at the center and connected to similar movies through derived relationship edges.

15. Qualitative Evaluation

The current project emphasizes explainability and usability rather than benchmark optimization, so evaluation is presented mainly as a qualitative case study.

First, the recommendation page shows that the system can identify coherent recommendation groups from both existing-user profiles and custom-input profiles. The explanations are consistent with the recommendation logic: recommended movies often share genres with the user's previous preferences and are also supported by similar users.

Second, the Movie Explorer page demonstrates that the derived graph relationships are meaningful. When a movie is selected, the resulting neighborhood typically contains films with overlapping genres or audience patterns, which suggests that the graph construction captures nontrivial structure in the data.

Third, the SQL Analytics page confirms that the relational layer produces sensible high-level summaries. Popular genres such as Drama, Comedy, and Action dominate rating counts, while the top-movie table is stabilized by a minimum-rating threshold to reduce noise from movies with too few observations.

Although this evaluation is not a formal benchmark, it shows that the tool produces outputs that are interpretable, coherent, and aligned with the intended use cases.

16. Why Both Data Structures Are Needed

A key design decision in this project is the use of both a relational data structure and a graph-based data structure.

The **relational layer** is the best choice for:

- normalized storage,
- joins across multiple tables,
- filtering and ranking,
- group-by aggregation,
- descriptive analytics.

The **graph layer** is the best choice for:

- local relationship exploration,

- nearest-neighbor style reasoning,
- user similarity,
- movie similarity,
- recommendation explanation.

Using only the relational layer would make it harder to present neighborhood structure and multi-hop relationships in an intuitive way. Using only the graph layer would make standard aggregation tasks unnecessarily awkward. The hybrid design therefore matches the strengths of each structure to the task it handles best.

17. Design Considerations

Several design decisions were important in implementation.

First, the project keeps the relational schema normalized. Genres are separated into their own table rather than being stored only as raw text, which supports cleaner joins and analytics.

Second, the recommendation scoring method is intentionally simple and interpretable. The goal of the project is not to implement a state-of-the-art recommender model, but to build a system whose outputs can be inspected and explained.

Third, the graph relationships are derived from observable behavior in the dataset. Because the dataset does not include explicit friendship or follow links, the application constructs edges from co-rating behavior, genre overlap, and shared tags. This is a reasonable design for the available data.

Fourth, the user interface is designed around the report requirements. The app includes dedicated pages for recommendation, analytics, schema visualization, and relationship exploration so that each core requirement can be demonstrated clearly.

18. Limitations and Future Work

The project has several limitations:

- The dataset is relatively small, so recommendation variety is limited.
- No demographic user information is available.
- Movie metadata is limited mainly to title, genres, tags, and external IDs.

- The recommendation score is heuristic and not tuned for formal benchmark performance.
- The graph layer is implemented in Python rather than an external graph database system.

There are several natural extensions for future work:

- replacing the current graph layer with Neo4j,
- using `links.csv` to pull richer movie metadata from IMDb or TMDB,
- adding embedding-based similarity or a vector database,
- evaluating recommendations with precision@k or recall@k,
- deploying the application as a public web app.

19. Conclusion

MovieLens Explorer demonstrates how relational and graph-based data structures can be combined in a single movie recommendation application. The relational layer supports storage, joins, filtering, and analytics, while the graph layer supports relationship discovery, similarity reasoning, and recommendation explanation. The final system satisfies the project requirement of including both a relational database element and a graph data element, while also producing a usable interactive application. Most importantly, the project shows that recommendation systems become more useful when they provide interpretable evidence rather than only ranked outputs.

References

[1] F. Maxwell Harper and Joseph A. Konstan, *The MovieLens Datasets: History and Context*, ACM Transactions on Interactive Intelligent Systems, 5(4), 2015.